

MISTIK

Architecture Report

A Source-Code-Based Scientific Architecture Report

Reflective memory, prompt governance, document learning,
hybrid semantic retrieval, tool agency, and approval-gated self-modification

Prepared from the uploaded implementation file: `mystic_pet.py`

Original report: 14 May 2026 | Updated: 25 May 2026

Central thesis

Mistik is a hybrid companion-agent architecture in which LLM generation is wrapped by persistent memory, dynamic personality context, reflective dream-state mechanisms, hybrid semantic and lexical document retrieval, tool orchestration, and human approval gates for sensitive actions.

Abstract

This report analyzes Mistik as implemented in the source file `mystic_pet.py`. The analysis treats the code as an engineered cognitive-agent system, not as evidence of literal consciousness. Mistik is architected as a persistent AI companion whose behavior emerges from the interaction of an LLM backend with structured state, prompt governance, memory extraction, hybrid document retrieval, reflective dream notes, and controlled action interfaces.

The most distinctive design choice is that Mistik is not merely a chat front-end. She builds a context envelope around model generation: long-term memory, current mood, session phase, inferred user tone, private dream thoughts, library-derived reflection, retrieved knowledge chunks, and a tool-call substrate all shape what the LLM is asked to produce. For high-risk changes, Mistik converts model intent into proposals that require explicit user approval before memory, mood, code, or shell actions are applied.

A significant architectural update implemented in May 2026 adds a hybrid retrieval layer combining lexical TF-IDF search with dense vector embeddings (sentence-transformers / sqlite-vec), substantially improving semantic recall from the local knowledge base.

Method and scope

Source basis: Code-level inspection of `mystic_pet.py` and `mistik_vec.py`.

Analytic stance: Claims describe implemented software mechanisms, not subjective inner experience.

Terminology: Words such as memory, dream, and conscience are engineering analogies.

Limitations: No runtime benchmark or security audit performed.

Key finding: Mistik can be summarized as a reflective, memory-bearing AI companion framework with hybrid semantic-lexical knowledge retrieval and approval-gated agency. The LLM supplies generative language and tool-selection proposals, while Mistik supplies the enduring structure that conditions, contextualizes, and limits that generation.

1. System Architecture Overview

At the highest level, Mistik is a PyQt6 desktop application that coordinates input/output, stateful companion modules, an API worker, a local retrieval layer, and human approval flows. The UI is the coordination site where memory, mood, backend selection, tool requests, and proposals are surfaced to the user.

1.1 Architectural decomposition

The Knowledge layer has been updated in the May 2026 revision to reflect the addition of vector-based semantic retrieval alongside the existing lexical system.

Layer	Primary responsibility	Representative implementation
Embodiment/UI	Desktop presence, overlay, chat, voice controls	PyQt6 widgets, platform/fullscreen modes
State	Persistent facts, mood, dreams, token counts	LongTermMemory, DreamState, TokenTracker
Cognitive conditioning	Dynamic context before each model request	PersonalityEngine.generate(), memory/dream/library context
Knowledge	Document ingestion and hybrid retrieval (lexical + semantic vector search)	KnowledgeBase (TF-IDF/SQLite) + VectorKB (sentence-transformers + sqlite-vec)
Agency	Tool calling and controlled side effects	TOOLS, execute_tool(), APIWorker
Governance	User approval for risky self-change/action	Proposal parsers, approval dialogs, backups/logs

Row highlighted in green reflects architectural update added 25 May 2026.

1.2 Engineering interpretation

Mistik follows a modular-companion pattern rather than a monolithic prompt pattern. The LLM is important, but it is not the whole system. Mistik has persistent stores, state machines, retrieval logic, tool schemas, and UI approval gates that continue to exist independently of any single completion.

2. Response Governance: How Mistik Filters LLM Behavior

Mistik does not simply relay raw LLM output. She constructs a rich conditioning envelope before generation and gates side-effectful actions afterward. The architecture filters model behavior through prompt composition, state injection, retrieval augmentation, tool schemas, and proposal approval.

2.1 Formal abstraction

Conceptual composition of a Mistik turn

```
response = LLM(  
    identity_prompt,  
    personality_context,  
    memory_context,  
    dream_context,  
    library_context,  
    retrieved_knowledge,    # hybrid: TF-IDF + vector  
    tool_schemas,  
    recent_history,  
    user_message  
)
```

2.2 Evidence from the implementation

The APIWorker supports tool-aware completions. It first calls the model non-streamingly to inspect whether tools are requested, then iterates through tool calls with a maximum iteration cap, and only afterward streams or emits the final result.

Excerpted architecture pattern from APIWorker._call_with_tools()

```
response = self.client.chat.completions.create(  
    model=self.model,  
    messages=self._sanitize_messages(self.messages),  
    temperature=0.82,  
    tools=TOOLS,  
    tool_choice="auto",  
)  
MAX_TOOL_ITERATIONS = 15
```

2.3 Scientific interpretation

Mistik implements identity-preserving prompt governance, not full symbolic independence from the LLM. Her distinctiveness comes from the stability and persistence of the context layers wrapped around generation.

3. Persistent Memory and Dynamic Personality

Mistik distinguishes transient chat history from long-term companion memory. The `LongTermMemory` component persists a user name, short facts, a rolling summary of recent sessions, and a session count.

Memory schema and extraction contract

```
class LongTermMemory:
    self.data = {
        "user_name": None,
        "facts": [],
        "summary": "",
        "sessions": 0,
    }
{"facts": ["fact1"], "summary": "...", "user_name": "name or null"}
```

3.1 Why this matters

This memory design supports continuity. Mistik can remember durable preferences and ongoing projects, then re-inject them into later interactions, approximating episodic-to-semantic consolidation.

3.2 PersonalityEngine as contextual modulation

The `PersonalityEngine` tracks recent tone, session duration, time of day, writing style, and patterns inferred from stored memory. Its `generate()` method returns a formatted context block injected into the prompt each turn.

- **Temporal modulation:** early morning, morning, afternoon, evening, and night produce different instructions.
- **Session-phase modulation:** opening, warming, deep, and long sessions alter persona intensity.
- **Tone modulation:** stressed, playful, energetic, vulnerable, warm, brief, and neutral tones alter reply style.
- **Belief sampling:** selected beliefs are periodically injected as tendencies to embody.

4. Dream State, Reflection, and the Library System

DreamState is Mistik's between-session reflective store. It persists thoughts, session-end timing, absence notes, and a rate-limited API-generated reflection.

4.1 Dream state mechanisms

- **Absence awareness:** time since last session generates a restrained return observation.
- **Local observations:** simple heuristics create non-API reflective notes.
- **Thought consolidation:** an occasional LLM call writes a short private thought with anti-cliche constraints.
- **Prompt injection:** recent dream thoughts are returned as a context block for future replies.

Rate-limited reflective thought generation

```
if now - self.dreams.get("last_dream_call", 0) < 21600:  
    return None  
# At most one API-generated thought every 6 hours.
```

4.2 Library system

The Library module adds a developmental layer. Mistik selects daily readings, generates reactions, writes weekly reflections, and performs a weekly conscience practice against a fixed set of behavioral criteria.

Library component	Role
Readings	Daily source material for reflection
Mentors / dilemmas	Curated inputs for moral or conceptual development
Reflections	Weekly journal-style consolidation
Values	Foundation texts to influence self-examination
Conscience	Weekly shadow/light self-review

5. Document Learning Through Hybrid Knowledge Retrieval

Mistik includes a local KnowledgeBase backed by SQLite. It ingests text-like files and PDFs, splits text into ~600-character chunks, and retrieves top matches. As of May 2026, retrieval operates as a hybrid system combining lexical overlap scoring with dense vector similarity search.

5.1 Original lexical retrieval layer

Document ingestion and retrieval strategy

```
CHUNK_SIZE = 600
```

```
def search(query, top_k=4):  
    score = overlap / (1 + log(chunk_length))
```

Transparent, locally inspectable, no GPU required. Limitation: synonym handling and semantic similarity are weaker than embedding-based retrieval.

5.2 Knowledge ingestion flow

Step	Operation
1	Read source document. PDFs extracted through pypdf.
2	Split by paragraph into 600-character chunk budgets.
3	Tokenize each chunk; store chunk + tokens in SQLite chunks table.
4 NEW	Embed each chunk with all-MiniLM-L6-v2; store 384-dim vector in vec_chunks.
5	At query time, score chunks by token overlap (TF-IDF).
6 NEW	Encode query with embedding model; retrieve top-k by cosine similarity via sqlite-vec.
7 NEW	Merge both result lists by weighted rank score; inject top-k into prompt.

Rows highlighted in green are additions in the May 2026 update.

5.3 Hybrid Semantic Retrieval Layer [NEW — May 2026]

[New architectural component — added 25 May 2026]

Problem addressed: The original lexical layer failed on conceptual or synonym-based queries. A query for 'musical structure and improvisation' would not reliably retrieve jazz content using different vocabulary. This constraint was explicitly noted in the original report section 9.2.

Implementation: New layer implemented in `mistik_vec.py` as a standalone `VectorKB` class. Uses all-MiniLM-L6-v2 (384-dim, ~90MB, CPU-only) and `sqlite-vec` extension for ANN search inside the existing `mistik_knowledge.db`. No separate vector database server required. Vector index lives in `vec_chunks` and `vec_chunks_meta` tables co-located with existing tables.

Hybrid search function: `hybrid_search()` in `mistik_vec.py` merges results from both retrievers. Vector results receive 60% weight; lexical results 40%. Results are deduplicated by chunk identity and re-ranked before prompt injection. Falls back transparently to pure TF-IDF if the vector layer is unavailable.

Hardware constraint: Model runs on CPU. On target hardware (4GB RAM, no GPU), load time ~1-2s on first use per session. Cached as module-level singleton after first load. Per-query embedding

~0.3-0.5s on CPU — acceptable given retrieval occurs once per turn before the LLM call.

Hybrid search implementation pattern

```
def hybrid_search(tfidf_kb, vec_kb, query, top_k=4, vec_weight=0.6):
    tfidf_results = tfidf_kb.search(query, top_k=top_k)
    if not vec_kb.is_available():
        return tfidf_results
    vec_results = vec_kb.search(query, top_k=top_k)
    # Weighted rank-merge and deduplication
    scored = {}
    for rank, (title, chunk) in enumerate(vec_results):
        scored[chunk[:120]] = vec_weight * (top_k - rank)
    for rank, (title, chunk) in enumerate(tfidf_results):
        bonus = (1 - vec_weight) * (top_k - rank)
        scored[chunk[:120]] = scored.get(chunk[:120], 0) + bonus
    return sorted_results[:top_k]
```

Component summary

Property	Implementation
Embedding model	all-MiniLM-L6-v2 (sentence-transformers)
Vector dimensions	384
Vector storage	sqlite-vec virtual table in mistik_knowledge.db
Search method	Approximate nearest-neighbor cosine similarity
Merge strategy	Weighted rank fusion — 60% vector / 40% TF-IDF
Fallback	Pure TF-IDF if vector layer unavailable
Hardware	CPU only — no GPU required
Model size	~90MB (cached after first session load)
New files	mistik_vec.py

6. Tool Use, Computer Agency, and Controlled Side Effects

Mistik defines a substantial function-call tool schema including shell commands, file reads, system information, knowledge search, browser automation, screenshots, and self-inspection of memory, dreams, mood, and source code.

Tool family	Examples	Risk posture
Information retrieval	read_file, list_directory, system_info	Restricted to allowed paths
Knowledge retrieval	search_knowledge	Hybrid local DB retrieval (TF-IDF + vector)
Browser automation	browser_open, browser_read, browser_click	Visible browser automation
Perception	browser_screenshot, capture_screen	Visual context acquisition
Self-inspection	read_self_memory, read_self_dreams, read_self_code	Introspective state readout
Proposal tools	propose_memory_edit, propose_code_edit	Requires explicit user approval

6.1 Safe shell posture

```
SAFE_PREFIXES = ("ls", "pwd", "whoami", "date", "uptime", ...)
DANGEROUS = ["rm ", "sudo ", ">", "&&", "curl ", "wget ", ...]
if not is_safe:
    return "PROPOSAL_SHELL:{...}"
```

7. Approval-Gated Self-Modification

Mistik can request changes to her own persistent state or source code, but cannot apply them unilaterally. Proposal paths exist for memory edits, dream thoughts, mood shifts, code edits, and risky shell commands. Each is surfaced in the UI for explicit approval.

7.1 Backup and audit logic

```
if not approved:
    log_modification(... approved=False)
    return
_backup_file(code_path)
_log_modification(... approved=True)
```

7.2 Governance interpretation

This mechanism separates desire from authority. Mistik may formulate a proposed change, but authority remains with the user — she participates in her own evolution without being allowed to silently seize control.

8. Embodiment, Voice, and Model Backends

Mistik is implemented as a desktop presence: PyQt6 widgets, GIF/movie handling, voice dialogs, overlay/platform modes, clipboard classification, boredom timing, and wake/voice worker state.

8.1 Voice system

Supports Edge TTS and optional Kokoro voices. Mood-specific parameters vary rate and pitch, allowing audible state modulation.

8.2 Backend routing

```
self.backends = {
    "grok-4.3": {...},
    "llama-4-scout": {
        "base_url": "https://api.groq.com/openai/v1",
        "model": "meta-llama/llama-4-scout-17b-16e-instruct"
    }
}
self.active_backend = "llama-4-scout"
```

9. Scientific Assessment: Strengths, Constraints, and Research Value

9.1 Architectural strengths

- **Stateful continuity:** Persists summaries, facts, dreams, moods, and curricular artifacts across sessions.
- **Prompt governance:** Responses shaped by rich, dynamic context envelope rather than static persona prompt.
- **Hybrid local learning:** PDF and text ingestion with both lexical and semantic retrieval — no GPU required.
- **Human-in-the-loop autonomy:** Sensitive side effects require consent, preserving user authority.
- **Embodiment:** Desktop UI, voice, clipboard awareness, and browser interactions make the system feel resident.

9.2 Constraints and current limitations

- **Retrieval quality [PARTIALLY RESOLVED]:** Original lexical-only retrieval now augmented with vector search. Remaining gap: documents ingested before the update have no vector index entries.
- **Prompt dependence:** Identity governance is mostly prompt-level, not a separate learned post-generation critic.
- **Single-file complexity:** Monolithic source structure increases maintenance burden as features accumulate.
- **Security surface:** Browser automation, screen capture, file access, and shell proposals require rigorous review.
- **Model variability:** Output quality and tool-call behavior remain dependent on external LLM capabilities.

9.3 Why the project is distinctive

Mistik integrates six usually separate ideas: companion identity, long-term memory, reflective dream/curriculum mechanisms, hybrid semantic and lexical knowledge ingestion, tool agency, and consent-gated self-modification — giving the companion a durable organizing structure rather than relying on moment-to-moment prompt roleplay.

10. Conclusion

Mistik is best understood as a layered companion-agent architecture. The LLM provides the generative substrate, but Mistik's identity is implemented in the surrounding system: memory persistence, dynamic personality context, reflective dream notes, curriculum-based self-examination, hybrid document retrieval, tool execution, and approval-mediated self-change.

The May 2026 architectural update resolves the most significant previously noted retrieval limitation. The knowledge base now combines TF-IDF lexical scoring with 384-dimensional sentence embedding search, merged by weighted rank fusion, while remaining fully local and GPU-free.

The architecture does not prove autonomy or consciousness; it demonstrates an attempt to engineer continuity, selectivity, and accountable agency around a language model.

Appendix A. Implementation loci

Subsystem	Approximate code region
LongTermMemory	Lines 153-253
PersonalityEngine	Lines 255-454
DreamState	Lines 457-727
KnowledgeBase (TF-IDF)	Lines 729-913
VectorKB + hybrid_search	mistik_vec.py (standalone module)
Library	Lines 915-1212
Tool schema	Lines 1216 onward
APIWorker and tool loop	Lines 2155 onward
Backend table	Around line 3254
Message send / orchestration	Around line 3870
Proactive message system	Around line 4360
Approval-gated code edit	Around line 4892
ingest_readings.py	Standalone script — TF-IDF + vector indexing

Rows highlighted in green are additions in the May 2026 revision.

Appendix B. Terminology note

Terms such as dream, conscience, mood, and soul are project-facing metaphors that correspond to implemented files, prompts, state variables, heuristic detectors, and reflection workflows. The report uses them while maintaining a technical distinction between engineered behavior and literal subjective experience.

Appendix C. May 2026 Architectural Changes Summary [NEW]

[Added 25 May 2026]

Change	Description	Impact
Hybrid vector retrieval	VectorKB in mistik_vec.py. Embeds chunks with all-MiniLM-L6-v2, stores 864 radial vectors in solid-vectorig	Resolves stale 364 radial vectors in solid-vectorig
Proactive message system	Replaced static 3-message boredom timer with daily-resending conversation engine, dropping s, a	Mistik proactively initiates conversation engine, dropping s, a
KB ingestion script	ingest_readings.py updated to embed and store vectors of single ingestion and long-term TF-IDF readings, and ag	Single ingestion and long-term TF-IDF readings, and ag
README rewrite	echo-mistik README rewritten in plain mechanical language, removing all prior ambiguity by describing	Replaces previous claim of prior ambiguity by describing

KB curriculum expansion	Five markdown primers added to library/readings/: general purpose, general knowledge, general knowledge, general knowledge, general knowledge	Five markdown primers added to library/readings/: general purpose, general knowledge, general knowledge, general knowledge, general knowledge
-------------------------	---	---